

Reference Production Architecture — Vacation Rental Marketplace

The system a listing experience of this kind would sit inside at marketplace scale. This is a design reference, not a description of the StayVista repository.

SCOPE

Everything on this page is a recommendation except one box. StayVista as submitted is a desktop front end with a frontend-held data layer and no backend at all — the single node outlined in green. It implements no service, database, queue or integration shown here. The diagram answers the assignment’s request for a production-scale architecture; it does not claim the clone implements it. Flows are explained in **docs/15-production-architecture.md**.

CLIENTS

Web — desktop

Server-rendered listing page, photo tour, lightbox

BUILT IN THIS REPOSITORY

Web — mobile

Out of scope: desktop-only brief (F1)

Native apps

iOS / Android, same public API

Admin & operations

Moderation, support, finance tooling

HTTPS

EDGE

DNS

Anycast, health-checked failover

CDN

Static assets and images; cache keys by variant

WAF & DDoS

Rule sets, bot mitigation, IP reputation

Load balancer & API gateway

TLS termination, routing, authn, rate limiting

internal, mutually authenticated

APPLICATION SERVICES — STATELESS, HORIZONTALLY SCALED

Listing

Listing read/write, media references

Search

Query, filter, rank, paginate

Availability

Calendars, blocks, minimum stay

Booking

Reservation lifecycle, idempotent

Pricing

Base, seasonal, fees, taxes, promos

Payment

Authorise, capture, refund, payout

User & auth

Identity, sessions, roles

Review

Submission, moderation, aggregation

Media

Upload, transcode, derivative variants

Notification

Templating, preferences, delivery

Recommendation

Similar stays, ranking signals

Host

Onboarding, calendar, payouts

Admin & moderation

Policy enforcement, takedowns

Geo

Geocoding cache, map tile proxy

synchronous reads and writes · published events

DATA

Primary relational DB

Per-service schemas; bookings strongly consistent

Read replicas

Listing and review reads; replica lag budgeted

Cache

Hot listings, pricing, session data; TTL + event bust

Search index

Denormalised documents, rebuilt from events

Object storage

Originals and derivatives; CDN origin

Analytics / event store

Append-only; warehouse and ML features

Availability store

Calendar state with transactional holds

Backups & PITR

Cross-region, restore-tested

ASYNCHRONOUS PROCESSING

Message broker / event bus

Durable topics, consumer groups, DLQs

Background workers

Retries with backoff; idempotent handlers

Search indexing pipeline

Listing and review events to index

Notification processing

Fan-out, batching, quiet hours

Media processing

Resize, compress, derivative generation

Recommendation pipeline

Feature build, offline scoring

EXTERNAL SYSTEMS

Payment provider

PSP; card data never touches our systems

Maps & geocoding

Tiles, forward and reverse geocoding

Email / SMS / push

Transactional delivery, webhooks for status

Identity provider

Social and enterprise sign-in (OIDC)

OBSERVABILITY

Logs

Structured, correlation id per request

Metrics & traces

RED/USE dashboards; distributed tracing

Alerting & SLOs

Error budgets, paging, runbooks

SECURITY

IAM & authorisation

Least privilege, per-service identity

Secrets & encryption

Managed secrets, TLS in transit, KMS at rest

Rate limiting & audit

Per-principal quotas; immutable audit log

PRINCIPAL FLOWS — DETAILED IN DOCS/15-PRODUCTION-ARCHITECTURE.MD

1. **Browse / search.** Client → CDN/WAF → API gateway → Search → search index → Listing → cache, then replica on miss → response. Cache and index absorb read traffic; the primary database is not on this path.

2. **Listing photos.** Client → CDN → object storage on miss. Images never transit an application service. StayVista links to the reference origin at runtime and packages nothing.

3. **Booking.** Client → gateway → Booking → Availability (transactional hold) → Pricing → Payment → PSP → booking database → event bus → notifications. Idempotency keys make a retried request safe; the hold is what prevents double-selling.
4. **Host listing update.** Host → Host/Admin → Listing → database → event bus → search re-index and media processing → CDN invalidation. Search is eventually consistent by design.

5. **Reviews.** Guest → Review → moderation → database → event pipeline → rating aggregates on the listing. Aggregates are recomputed asynchronously, never in the write path.